

JavaScript Advanced Fundamentals

Destructuring se le kar Closures aur Hoisting tak — core concepts, real-world scenario-based practice questions ke sath, terminal mein run karne ke liye.

01 Destructuring, Spread/Rest, Default Params, Template Literals

02 ES Modules — Import / Export & File Organisation

03 Promises, Async/Await & Error Handling

04 Fetch API — GET/POST, Headers, JSON, Errors

05 Immutability — Shallow vs Deep Copy

06 Pitfalls — this, Closures & Hoisting

01 Destructuring, Spread/Rest, Default Params, Template Literals

DESTRUCTURING

Array/object se values seedha variables mein nikalna, bina index ya key baar baar likhe.

```
const user = { name: "Ali", age: 24, city: "Lahore" };
const { name, city } = user;

// rename karna ho to
const { name: userName } = user;
```

SPREAD VS REST

Spread = phailana (unpack karke copy/merge banana). **Rest** = sametna (multiple values ko array mein collect karna).

```
// Spread - copy/merge
const obj2 = { ...obj1, z: 3 };

// Rest - function parameters mein collect
function sum(...numbers) {
  return numbers.reduce((a, n) => a + n, 0);
}
```

DEFAULT PARAMETERS & TEMPLATE LITERALS

```
function greet(name = "Guest") {
  console.log(`Hello, ${name}!`);
}
```

● PRACTICE QUESTIONS

Q1. Order Receipt EASY

Tumhare paas ek order object hai: `{ id: 101, item: "Laptop", price: 85000, buyer: { name: "Ali", city: "Lahore" } }`.

Destructuring use karke **item**, **price**, aur nested **buyer.name** nikalo, phir template literal se print karo: `"Order #101: Ali ne Laptop khareeda (Rs. 85000)"`

Q2. Cart Merge EASY

Do arrays hain: `oldCart = ["pen", "notebook"]` aur `newItems = ["eraser"]`.

Spread operator se dono ko merge karke ek naya array `updatedCart` banao (originals ko mutate kiye bagair).

Q3. Expense Splitter MEDIUM

Ek function `splitBill(...amounts)` banao jo rest parameter se koi bhi tadaad mein numbers le, total nikale, aur per-head average return kare (2 friends fixed maan lo).

Test karo: `splitBill(500, 300, 700)` → total aur average dono console pe print ho.

Q4. Profile Card Generator MEDIUM

Ek function `buildProfile(name, role = "Intern", ...skills)` banao.

Default param, rest param, aur template literal ka use karke ek formatted profile string return karo, jaise: `"Ali - Intern - Skills: JS, Node, React"`

02 ES Modules — Import / Export & File Organisation

Node.js mein ES Modules enable karne ke liye `package.json` mein `"type": "module"` add karo, ya file ko `.mjs` rakho.

NAMED VS DEFAULT EXPORTS

```
// math.js
export const add = (a, b) => a + b;
export default function multiply(a, b) { return a * b; }

// main.js
import multiply, { add } from "./math.js";
```

BEST PRACTICE

Har module ek single responsibility rakhe (e.g. `db.js`, `routes/user.js`, `utils/validate.js`) — FastAPI ke router-based structure jaisa mental model.

● PRACTICE QUESTIONS

Q1. Utility Toolkit EASY

Ek `utils.js` file banao jisme `isEmail(str)` aur `isStrongPassword(str)` named exports hon.

Dusri file `main.js` mein donon import karke test karo ek fake signup form validate karke.

Q2. Logger Module MEDIUM

Ek `logger.js` file mein default export `logInfo(msg)` aur named export `logError(msg)` dono banao.

`main.js` mein dono ko sahi syntax se import karke ek chota "server events" simulation likho (start, error, stop).

Q3. Mini Project Structure MEDIUM

Ek chota CLI project socho jisme `models/user.js`, `services/userService.js`, aur `index.js` hon.

`user.js` se ek `createUser()` function export karo, `userService.js` se usay import/use karo, phir `index.js` se poora flow chalao.

03 Promises, Async/Await & Error Handling

PROMISE BASICS

```
function delay(ms) {  
  return new Promise((resolve, reject) => {  
    setTimeout(() => resolve(`${ms}ms baad`), ms);  
  });  
}
```

ASYNC/AWAIT + TRY/CATCH

```
async function run() {  
  try {  
    const result = await delay(1000);  
    console.log(result);  
  } catch (err) {  
    console.error("Error:", err.message);  
  }  
}
```

SEQUENTIAL VS PARALLEL

Agar tasks independent hain to `Promise.all([...])` use karo, sequential `await` se zyada fast hoga.

● PRACTICE QUESTIONS

Q1. Food Delivery ETA EASY

Ek Promise-based function `deliverOrder(orderId)` banao jo 2 second baad resolve ho "Order #id delivered" message ke sath.

`async/await` aur `try/catch` ke sath call karo aur result print karo.

Q2. Payment Gateway Simulation MEDIUM

Ek function `processPayment(amount)` banao jo agar `amount <= 0` ho to reject kare ("Invalid amount"), warna 1 second baad resolve ho.

Dono cases (valid aur invalid amount) test karo, aur error `catch` block mein handle karo.

Q3. Multiple API Calls Parallel MEDIUM

3 fake async functions banao: `getUser()`, `getOrders()`, `getNotifications()` — har ek alag delay ke sath resolve ho.

`Promise.all()` use karke teeno ko parallel chalo aur total time compare karo sequential `await` se.

Q4. Retry Logic MEDIUM

Ek function `unreliableServer()` banao jo random 50% chance se reject ho jaye.

Ek `retry(fn, attempts)` wrapper likho jo fail hone par dobara try kare (max 3 baar), `async/await` ke sath.

04 Fetch API — GET/POST, Headers, JSON, Errors

Node 18+ mein `fetch` built-in hai — terminal mein directly chalega, koi package chahiye nahi.

```
const res = await fetch(url);
if (!res.ok) throw new Error(`HTTP ${res.status}`);
const data = await res.json();
```

GOTCHA

`fetch` sirf network-level errors par reject karta hai. 404/500 par bhi `resolve` hi karega — isliye `response.ok` khud check karna zaroori hai.

● PRACTICE QUESTIONS

Q1. Weather Checker EASY

Public API `https://jsonplaceholder.typicode.com/users/1` se GET request karo.

Response ko JSON parse karke sirf `name` aur `email` print karo. Agar request fail ho to friendly error message do.

Q2. New User Signup MEDIUM

POST request bhejo `https://jsonplaceholder.typicode.com/users` par, body mein `{ name, email }` JSON ke sath.

`headers` mein `Content-Type: application/json` set karo, aur server ka response console pe print karo.

Q3. Invalid Endpoint Handling MEDIUM

Jaan-boojh kar ek galat URL (jo 404 de) fetch karo.

`response.ok` check karke proper error throw karo aur `try/catch` mein clean message print karo (na ke raw crash).

Q4. Multiple Users Fetch MEDIUM

IDs `[1, 2, 3]` ke liye teen users ka data fetch karna hai.

`Promise.all()` aur `map` use karke parallel fetch karo, sab names ek array mein print karo.

05 Immutability — Shallow vs Deep Copy

```
// Shallow copy – nested object ka sirf reference copy hota hai
const shallowCopy = { ...original };

// Deep copy – nested levels tak independent
const deepCopy = structuredClone(original);
```

YAAD RAKHO

`map`, `filter`, `reduce`, `slice` naya array return karte hain (immutable). `push`, `splice`, `sort` original ko mutate karte hain.

● PRACTICE QUESTIONS

Q1. Todo List Update EASY

Ek array `todos = [{id:1, done:false}, {id:2, done:false}]` hai.

Bina original array mutate kiye, id=1 wale todo ka `done` true karo (spread + map use karo) aur naya array print karo.

Q2. Shallow Copy Bug EASY

Ek object `student = { name: "Ali", grades: { math: 80 } }` hai. Shallow copy banao `{ ...student }` se.

Copy mein `grades.math` change karo, phir dikhao ke original bhi change ho gaya — samjhao kyun.

Q3. Fixing the Bug with Deep Copy MEDIUM

Wahi `student` object le lo jo Q2 mein tha.

`structuredClone()` use karke deep copy banao, is baar copy modify karne par original safe rehna chahiye — confirm karo `console.log` se.

06 Pitfalls — this, Closures & Hoisting

THIS BINDING

Regular function mein `this` call-site par depend karta hai. Arrow function apna `this` nahi banata — outer scope ka use karta hai.

CLOSURES

Function apne outer scope ki variables yaad rakhta hai, chahe outer function khatam ho chuki ho.

HOISTING

`var` hoist hoke `undefined` ban jata hai. `let` / `const` Temporal Dead Zone mein rehte hain. Function declarations poori hoist hoti hain.

CLASSIC BUG

`for (var i = 0; i < 3; i++) setTimeout(() => console.log(i), 100)` → prints `3, 3, 3`. Fix: `let` use karo.

● PRACTICE QUESTIONS

Q1. Broken Button Handler EASY

Ek object `counter = { count: 0, increment: function(){...} }` hai jisme `setTimeout` ke andar regular function use hone ki wajah se `this.count` update nahi ho raha.

Bug fix karo arrow function use karke, taake `this` sahi object ko refer kare.

Q2. Bank Account with Private Balance MEDIUM

Ek `createAccount(initialBalance)` function banao jo closure use karke `deposit(amount)` aur `withdraw(amount)` functions return kare.

Balance directly access na ho sake (private rahe), sirf returned functions se change ho.

Q3. Loop of Timers Bug EASY

Ek loop `for (var i = 1; i <= 3; i++) setTimeout(() => console.log("Ticket #" + i), i * 100)` galat output de raha hai.

`var` ko `let` se replace karke bug fix karo, aur pehle wala (galat) aur baad wala (sahi) dono output compare karo.

Q4. Hoisting Trap MEDIUM

Ek chota script likho jisme `console.log(a)` pehle chale, phir `var a = 5` ho.

Predict karo output kya aayega (before running), phir run karke confirm karo. Same experiment `let` ke sath karo aur error dekho.

Tip: har question ko pehle bina dekhe khud solve karne ki koshish karo, phir Node.js terminal mein `node file.js` se run karke verify karo.